



Imersão GenAI para Desenvolvedores

Módulo 4

Preâmbulo: Vector DB, Chunking e RAG

Versão: 1.1

Sumário

I	Exercício 1: Vector Database	2
II	Exercício 2: Chunking	4
III	Exercício 3: Chatbot especializado	6
IV	Entrega e Avaliação entre Pares	8

Capítulo I

	Exercício : 01
Vector Database	
Pasta de entrega : <i>ex01/</i>	
Arquivos para entregar : <code>chroma_db.py</code>	
Funções ou bibliotecas autorizadas : Biblioteca padrão, <code>sentence_transformers</code> , <code>chromadb</code>	

O conceito de vector database é fundamental quando trabalhamos com grandes coleções de textos e embeddings. Diferente do exercício anterior, onde os embeddings eram mantidos em memória, um vector database permite persistir, indexar e buscar rapidamente os vetores de forma escalável, mesmo em **datasets** grandes.

1. Utilizando a biblioteca [ChromaDB](#) e `sentence_transformers`, crie um programa que:
 - Armazene o conteúdo de cada frase do arquivo `orbit_motordrones.txt` em uma collection do ChromaDB.
 - Retorne as *n* frases mais relevantes para uma pergunta feita pelo usuário.
 - Exibir os resultados com o score de similaridade.
2. Ao seu critério, você pode definir o valor de *n* (número de frases a serem retornadas), e também aplicar um **threshold** (limiar mínimo de similaridade) para filtrar os resultados após a query. Isto pode ajudar a melhorar a relevância das respostas, mesmo que não retorne resultados para perguntas que não tenham correspondência suficiente.
3. O comportamento esperado é o mesmo do exercício anterior, mas agora utilizando um banco de dados vetorial para armazenar e recuperar os embeddings.

```
?> python3 chroma_db.py "Qual o prazo de entrega do orbit family?"
---- Linhas relevantes recuperadas: ----
- As primeiras entregas do Orbit Family estão previstas para logo após o início das pré-vendas em maio de 2026.
- O lançamento do Orbit Family está programado para o próximo semestre, prometendo uma revolução no mercado de mobilidade elétrica.
- As entregas do Orbit Urban estão previstas para ocorrer em 12 a 16 semanas após a confirmação do pedido.
```

As primeiras entregas do Orbit Family estão previstas para logo após o início das pré-vendas em maio de 2026.



Utilize um banco de dados persistente (`chromadb.PersistentClient`) e carregue os dados no banco de dados apenas na primeira vez que o programa for executado.

Capítulo II

	Exercício : 02
Chunking	
Pasta de entrega : <i>ex02/</i>	
Arquivos para entregar : chunking.py	
Funções ou bibliotecas autorizadas : Biblioteca padrão, sentence_transformers , chromadb	

A armazenagem em um banco de dados vetorial e a criação de embeddings é uma tarefa relativamente simples quando se utiliza dados individualizados, como linhas de texto. No entanto, quando se trabalha com documentos longos ou conjuntos de dados complexos, pode ser necessário pré-processar seus dados, dividindo o texto em partes menores para garantir que cada parte seja processada de forma eficaz.

Existem diversas estratégias para dividir textos, como por parágrafos, frases, ou até mesmo utilizando [técnicas de chunking](#) que consideram o contexto e a semântica do texto.

Neste exercício, vamos implementar a técnica de janela deslizante (sliding window), que cria pedaços de texto sobrepostos, permitindo que cada pedaço mantenha parte do contexto do anterior.

1. Crie uma função `def sliding_window_chunking(text: str, window_size: int, step_size: int) -> list[str]:` que divida um texto em pedaços (chunks) utilizando a técnica de janela deslizante (sliding window). A função deve receber o texto completo, o tamanho da janela (número de palavras por chunk) e o tamanho do passo (quantidade de palavras que a janela avança a cada iteração).
2. A função deve retornar uma lista de strings, onde cada string é um chunk do texto original.
3. Exemplo de funcionamento para a função (chunks pequenos para demonstrar o conceito):

```
text = """
A Orbit Motordrones é uma empresa inovadora que está revolucionando o mercado de mobilidade urbana
com seus motodrones de alta performance. Fundada em 2024, a empresa tem como missão oferecer
soluções de transporte aéreo pessoal que sejam seguras, eficientes e acessíveis.
"""
chunks = sliding_window_chunking(text, window_size=10, step_size=5)
```

```
for chunk in chunks:
    print(chunk)

# Saída esperada:
A Orbit Motordrones é uma empresa inovadora que está revolucionando
empresa inovadora que está revolucionando o mercado de mobilidade urbana
o mercado de mobilidade urbana com seus motodrones de alta
com seus motodrones de alta performance. Fundada em 2024, a
performance. Fundada em 2024, a empresa tem como missão oferecer
empresa tem como missão oferecer soluções de transporte aéreo pessoal
soluções de transporte aéreo pessoal que sejam seguras, eficientes e
que sejam seguras, eficientes e acessíveis."
acessíveis."
```



A técnica de janela deslizante é especialmente útil para garantir que o contexto seja mantido entre os chunks, mantendo uma sobreposição entre eles, o que pode melhorar a qualidade dos embeddings gerados posteriormente.



Neste exercício, utilizamos chunks pequenos para demonstrar o conceito. Em uma aplicação real, recomenda-se utilizar chunks maiores para preservar mais contexto. O tamanho do chunk e o passo devem ser ajustados conforme a especificidade dos textos.

Capítulo III

	Exercício : 03
Chatbot especializado	
Pasta de entrega : <i>ex03/</i>	
Arquivos para entregar : <code>process_pdfs.py</code> , <code>chatbot.py</code>	
Funções ou bibliotecas autorizadas : Sem restrições	

Neste exercício, você irá utilizar a IA para criar um chatbot especializado que utiliza um banco de dados vetorial para responder perguntas sobre um conjunto de currículos.

1. Crie um programa `process_pdfs.py` que:
 - Leia os arquivos PDF de uma pasta chamada **currículos** (arquivos disponíveis na intranet).
 - Extraia o texto de cada PDF.
 - Divida o texto em chunks utilizando a técnica de janela deslizante (sliding window).
 - Crie embeddings para cada chunk.
 - Armazene os embeddings em um banco de dados vetorial (ChromaDB), juntamente com metadados que identifiquem a quem pertence cada chunk (por exemplo, nome do candidato, nome do documento original, etc.).
2. Crie um programa `chatbot.py` que permita fazer perguntas sobre os currículos processados:
 - O programa deve receber uma pergunta do usuário.
 - Recuperar os chunks mais relevantes do banco de dados vetorial.
 - Utilizar um modelo de linguagem para gerar uma resposta baseada na pergunta e nos chunks recuperados.

- Exibir a resposta ao usuário.
3. Seu chatbot deve ser capaz de responder perguntas como 'quais candidatos possuem experiência em python'



Envie os metadados, junto com os chunks recuperados para o modelo de linguagem, para que ele possa fornecer respostas mais contextuais e precisas.



Não é necessário implementar memória. Seu foco deve ser no armazenamento e recuperação de informações relevantes para responder perguntas, baseadas em um conjunto de dados específico.

Capítulo IV

Entrega e Avaliação entre Pares

- Entregue seu projeto em seu repositório *Git* disponível na página do projeto na intranet.
- Apenas o trabalho dentro do seu repositório será avaliado durante a defesa. Não hesite em verificar os nomes de seus arquivos e pastas para garantir que estejam corretos.
- No horário da avaliação, o avaliador se dirigirá à estação de trabalho do aluno a ser avaliado para realizar os testes. Um clone do repositório deverá ser realizado em uma nova pasta, e estes são os arquivos que serão avaliados.